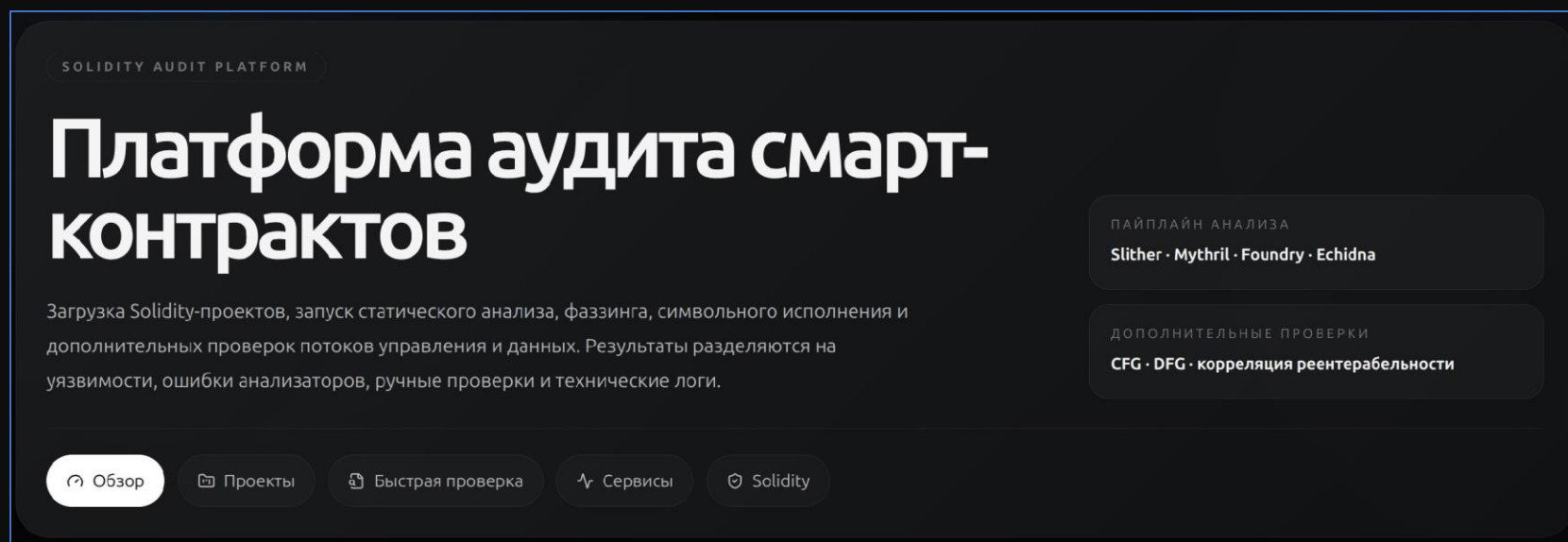


SOLIDITY AUDIT PLATFORM

Автоматизированный анализ Solidity-проектов перед публикацией в blockchain-сети: статический анализ, символьное исполнение, fuzzing, тесты и проверка потоков управления.



Выполнили:
Матвеев Илья
Баркова Александра
Зиновьев Тимофей

Безопасность контракта начинается до деплоя.

Ошибка в смарт-контракте становится частью сети

Смарт-контракты управляют активами, правами доступа, токенами, резервами и логикой протоколов. После деплоя ошибка может привести не просто к сбою сервиса, а к потере средств, блокировке активов или захвату управления контрактом.

Executive Summary

В обычном backend ошибку можно закрыть патчем. В blockchain-системе уязвимость часто становится публичной, воспроизводимой и доступной для атаки любому участнику сети.

Reentrancy

Повторный вход в функцию до обновления состояния.

tx.origin

Фишинговая авторизация через промежуточный контракт.

Нарушение инвариантов

Внутренний учет перестает соответствовать реальному балансу.

Solidity Audit Platform — pre-audit слой для Solidity-проектов

1. Загрузка проекта .sol или .zip

Платформа принимает одиночный .sol файл или .zip архив проекта, запускает несколько независимых анализаторов и собирает результаты в единый отчет.

2. Запуск pipeline Slither, Mythril, Foundry, Echidna, CFG/DFG

Executive Summary

Разработчик получает не разрозненные выводы инструментов, а структурированную картину рисков: уязвимости, ошибки анализаторов, ручные проверки, графы и технические логи.

3. Нормализация findings. Severity, rule, tool, location, message

4. Отчет и демонстрация риска

Один запуск — несколько взглядов на безопасность контракта.

Это blockchain-проект, потому что он защищает blockchain-логику

Платформа не создает собственную сеть и не вводит новый consensus. Ее blockchain-сущность заключается в другом: она **анализирует код**, который будет исполняться в EVM-сетях и управлять on-chain состоянием. Solidity Audit Platform работает до деплоя, но ее **результат напрямую влияет на безопасность будущего контракта** в Ethereum, Polygon, BNB Chain, Arbitrum, Optimism, Base и других EVM-совместимых сетях.

Executive Summary

Off-chain анализ → On-chain безопасность

Smart contracts

Анализируются контракты, которые будут исполняться в EVM.

EVM-совместимость

Поддерживаются Solidity-проекты и Foundry-структура.

Security before mainnet

Ошибки ищутся до публикации в testnet/mainnet.

Атаки на смарт-контракты повторяются — их нужно ловить системно

Reentrancy

Контракт отправляет средства внешнему адресу до обновления баланса. Атакующий контракт повторно вызывает `withdraw` и использует промежуточное состояние.

Executing contract

Авторизация через tx.origin

Владелец вызывает вредоносный контракт, а тот вызывает целевой кошелек. Проверка `tx.origin == owner` проходит, хотя непосредственный вызов сделал атакующий контракт.

Нарушение учета балансов

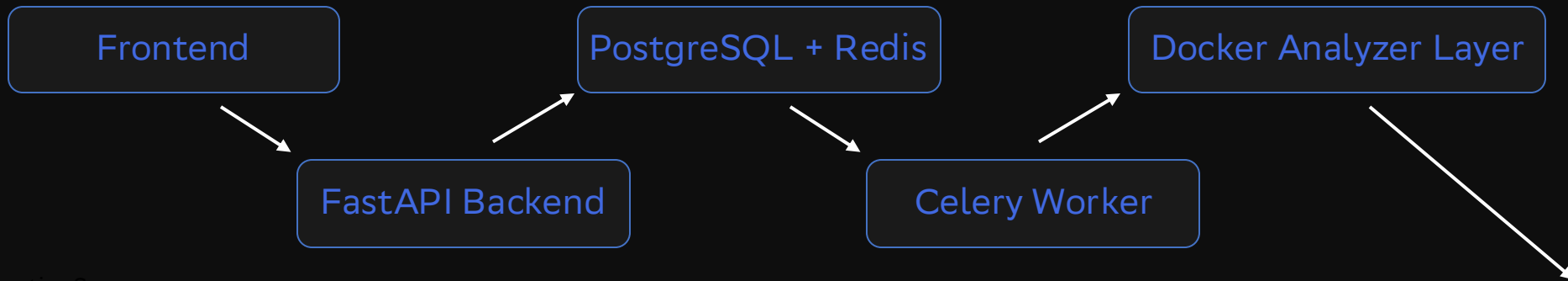
Внутренний mapping расходится с реальным балансом контракта. Пользовательские депозиты, резервы или `total supply` становятся неконсистентными.

Слабая случайность

Использование `block.timestamp`, `block.prevrandao` или других предсказуемых параметров для лотерей, NFT или игровых механик.

Платформа не просто ищет строки кода — она сопоставляет результаты разных анализаторов и помогает понять, почему участок опасен.

Архитектура: API, очередь задач и изолированные анализаторы



Executive Summary

Backend отвечает за загрузку проектов, запуск анализа, хранение результатов и выдачу API. Worker выполняет тяжелые проверки асинхронно.

Анализаторы запускаются в отдельных Docker-контейнерах, чтобы изолировать зависимости и пользовательский код.

Slither

Basic Scanner

Mythril

CFG / DFG

Foundry

Reentrancy
Correlation

Echidna

Manual Checklist

Анализатор безопасности сам должен быть безопасным

Пользовательский Solidity-код нельзя считать доверенным. Даже если платформа не исполняет контракт в реальной сети, она обрабатывает архивы, зависимости, конфиги и вывод внешних инструментов.

Read-only mount

Проект монтируется только для чтения.

Нет сети для analyzer runtime

Инструменты не должны обращаться наружу во время анализа.

Лимиты CPU / RAM / процессов

Ошибочный или тяжелый анализ не должен ломать всю систему.

Изоляция через Docker

Каждый тяжелый анализатор запускается отдельно.

Безопасная распаковка ZIP

Проверка путей, лимиты файлов и размера.

Pipeline объединяет разные методы поиска уязвимостей

Статический анализ

Slither и Basic Scanner находят опасные конструкции, reentrancy-паттерны, tx.origin, .call, delegatecall, selfdestruct, weak randomness.

Динамическая и символьная проверка

Foundry запускает сборку и тесты. Mythril исследует пути выполнения и ищет SWC-классы проблем.

Инварианты и графы

Echidna проверяет свойства через fuzzing. CFG/DFG фиксируют внешние вызовы, чтение и запись состояния. Reentrancy correlation связывает эти факты в отдельный риск.

Результат: единая таблица findings вместо разрозненных логов инструментов.

Перейдем к демонстрации

Executive Summary



Результат: единая таблица findings вместо разрозненных логов инструментов.

Платформа снижает стоимость ошибки до деплоя

Solidity Audit Platform помогает перенести часть security-проверок на ранний этап разработки. Команда может запускать анализ до testnet/mainnet, исправлять очевидные ошибки, проверять инварианты и готовить проект к ручному аудиту. Это особенно важно для Web3-команд, где одна ошибка в контракте может повлиять на средства пользователей и доверие к протоколу.

Executive

Меньше blind spots

Разные анализаторы
смотрят на код с
разных сторон.

Быстрее pre-audit

Разработчик получает первичную
картину риска без ручного запуска
всех инструментов.

Готовность к CI/CD

Pipeline можно развить в
GitHub Actions, CLI и
автоматический security gate.

Результат: единая таблица findings вместо разрозненных логов инструментов.

Проверять контракт после атаки поздно. Проверять нужно до деплоя.

Спасибо за внимание!

Executive Summary

Результат: единая таблица findings вместо разрозненных логов инструментов.